Министерство образования и науки Российской Федерации

Уральский федеральный университет
имени первого Президента России Б.Н.Ельцина

Е.И. Кац, А.М. Китаев

КОМПЬЮТЕРНАЯ ГРАФИКА

Электронное текстовое издание

Методические указания к лабораторным работам по курсу «Компьютерная графика» для студентов дневной формы обучения направлений 090301 «Информатика и вычислительная техника» и 090303 «Прикладная информатика».

Подготовлено кафедрой «Информационные технологии и автоматизация проектирования»

Определены формулировки заданий и описаны подходы к их решению с обоснованием их рациональности. Приведены некоторые неявные вопросы и ответы на них.

Екатеринбург

2015

ОГЛАВЛЕНИЕ

Оглавление **2**

1. Лабораторная работа № 1 **3**

1.1. Постановка задачи 3

1.2. Проектирование программы 3

1.2.1. Формат файла сцены 3

1.2.2. Считывание из файла 4

1.2.3. Структура данных программы 5

1.3. Неочевидные вопросы 10

2. Лабораторная работа № 2 **11**

2.1. Постановка задачи 11

2.2. Проектирование программы 11

2.2.1. Формат файла сцены 11

2.2.2. Реализация алгоритма Робертса 12

2.3. Возможные вопросы 14

3. Лабораторная работа № 3 **15**

3.1. Постановка задачи 15

3.2. Проектирование программы 15

3.2.1. Формат файла сцены 15

3.2.2. Реализация алгоритма Z-буфера 16

3.3. Возможные вопросы 18

4. Лабораторная работа № 4 **19**

4.1. Постановка задачи 19

4.2. Проектирование программы 19

ГЛАВА 1. ЛАБОРАТОРНАЯ РАБОТА № 1

1.1. Постановка задачи

Написать программу, которая:

- загружает из текстового файла трёхмерную сцену, состоящую из отрезков;
- отображает трёхмерную сцену (проекцию сцены на плоскость XY, ось X направлена вправо, Y – вверх, Z – на наблюдателя, начало координат расположено в центре области отображения);
- осуществляет над сценой аффинные преобразования: параллельный перенос, масштабирование, поворот вокруг осей координат и косой сдвиг;
- осуществляет над сценой ОПП[1] с заданным параметром;
- параметры преобразований задаются пользователем;
- осуществляет вписывание сцены в область отображения без искажений (одинаковое масштабирование по всем осям, отсутствие вращений).

1.2. Проектирование программы

1.2.1. Формат файла сцены

Рассмотрим задачу описания трёхмерной сцены в текстовом файле. Так как сцена состоит из отрезков, можно, к примеру, задавать каждый отрезок шестью координатами, по три координаты для каждого конца, x_1 y_1 z_1 x_2 y_2 z_2, разделённые пробелом. Каждый отрезок может быть записан в своей строке. Тогда содержание файла будет примерно таким:

```
0.0 0.0 0.0 10.0 12.0 15.0
10.0 12.0 15.0 10.0 12.0 0.0
```

Ещё один способ задать сцену из отрезков – отделить геометрическое описание точек в пространстве от топологии (связей между ними). В таком случае, содержание файла для сцены из первого примера будет выглядеть так:

[1] ОПП – одноточечное проективное преобразование

```
0.0 0.0 0.0
10.0 12.0 15.0
10.0 12.0 0.0
0 1
1 2
```

В начале файла описаны точки в каждой строке по три координаты. Координаты разделены пробелами. Далее в каждой строке по одному записаны отрезки. Каждый отрезок описывается двумя порядковыми номерами точек, соответствующих концам отрезка. Такой вариант описания сцены является более удобным, так как в геометрических фигурах каждая вершина часто является концом нескольких отрезков.

1.2.2. Считывание из файла

Для упрощения считывания из файла трёхмерной сцены можно ввести некоторые изменения в ранее предложенные форматы файла, а именно – ввести «блоки» точек и топологии. Содержание файла с блоками может выглядеть так:

```
3
0.0 0.0 0.0
10.0 12.0 15.0
10.0 12.0 0.0
2
0 1
1 2
```

Как видно из приведённого примера, к предыдущему описанию добавлено две строки. В первой указано количество точек, записанных в файле, во второй – количество отрезков. Такой подход позволяет не тратить время на анализ информации в строке, а заранее предполагать что именно, точка или отрезок, описаны в строке. Программа может прочитать из первой строки количество точек, затем повторить считывание точек из последующих строк указанное количество раз. Аналогично программа поступит и с «блоком» отрезков. Конечно, при та-

ком подходе нужно условиться, что количества точек и отрезков всегда указаны верно, в противном случае программа не сможет прочитать файл сцены.

1.2.3. Структура данных программы

Необходимо определиться со структурой данных, которая будет хранить трёхмерную сцену. В зависимости от порядка обработки данных структура хранимых переменных может меняться.

Самый примитивный класс, с которым необходимо работать,– это класс точки. Лучше всего хранить точку в виде массива из четырёх чисел с плавающей запятой. Таким образом, можно хранить однородные координаты точки и иметь доступ к каждой из координат по её номеру в массиве.

$$\begin{bmatrix} x & y & z & h \end{bmatrix}$$

Данные о геометрии и топологии предлагается хранить таким же образом, как и в файле: точки хранить в отдельном массиве, для каждого отрезка хранить номера его концов в массиве точек.

Разделять информацию о точках с информацией об отрезках более разумно, так как: работа с геометрической информацией отделена от работы с топологической (алгоритмы преобразования работают с координатами точек), объём используемой памяти обычно меньше (так как устраняется дублирование геометрической информации о точках при описании геометрических фигур).

Матрицы преобразования Матрицы преобразования отличаются для точек, представленных в виде матрицы строки и матрицы столбца. Условимся, что точки представлены в виде матрицы строки. Тогда матрицы преобразования будут выглядеть так[2]:

[2]См. Роджерс Д. «Математические основы машинной графики»

$$\begin{bmatrix} a & b & c & p \\ d & e & f & q \\ h & i & j & r \\ l & m & n & s \end{bmatrix},$$

где a, e, j отвечают за масштабирование соответственно по осям x, y, z; l, m, n отвечают за параллельный перенос соответственно по осям x, y, z

Матрица параллельного переноса на вектор $\vec{a}\,(x, y, z)$:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ x & y & z & 1 \end{bmatrix}.$$

Матрица масштабирования по оси X:

$$\begin{bmatrix} k_x & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Матрица масштабирования по оси Y:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & k_y & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Матрица масштабирования по оси Z:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & k_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Матрица поворота вокруг оси X на угол α:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\alpha & \sin\alpha & 0 \\ 0 & -\sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Матрица поворота вокруг оси Y на угол α:

$$\begin{bmatrix} \cos\alpha & 0 & -\sin\alpha & 0 \\ 0 & 1 & 0 & 0 \\ \sin\alpha & 1 & \cos\alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Матрица поворота вокруг оси Z на угол α:

$$\begin{bmatrix} \cos\alpha & \sin\alpha & 0 & 0 \\ -\sin\alpha & \cos\alpha & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Матрица косого сдвига оси X по оси Y с коэффициентом k:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ k & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Матрица косого сдвига оси X по оси Z с коэффициентом k:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ k & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Матрица косого сдвига оси Y по оси X с коэффициентом k:

$$\begin{bmatrix} 1 & k & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Матрица косого сдвига оси Y по оси Z с коэффициентом k:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & k & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Матрица косого сдвига оси Z по оси X с коэффициентом k:

$$\begin{bmatrix} 1 & 0 & k & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Матрица косого сдвига оси Z по оси Y с коэффициентом k:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & k & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Матрица ОПП по оси X с фокусным расстоянием f_x:

$$\begin{bmatrix} 1 & 0 & 0 & \frac{1}{f_x} \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Матрица ОПП по оси Y с фокусным расстоянием f_y:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & \frac{1}{f_y} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Матрица ОПП по оси Z с фокусным расстоянием f_z:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \frac{1}{f_z} \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Порядок обработки. Существует два варианта порядка обработки точек:

1. Хранение исходной сцены, обрабатываемой через итоговое преобразование перед отрисовкой, итоговое преобразование – это матрица, являющаяся результатом произведения матриц всех преобразований в соответствующем порядке.
2. Хранение преобразованной сцены с последовательным применением преобразований.

1.3. Неочевидные вопросы

1. Для ОПП можно задавать фокусное расстояние или непосредственно величину соответствующего элемента матрицы преобразования. Что именно используется в программе – часто задаваемый вопрос.
2. **Сколько существует разных простых косых сдвигов?** Существует шесть косых сдвигов: x/y (x по y) – элемент d матрицы преобразования, x/z – элемент h, y/x, y/z, z/x, z/y

ГЛАВА 2. ЛАБОРАТОРНАЯ РАБОТА № 2

2.1. Постановка задачи

Дополнить возможности программы первой лабораторной работы возможностями задания в исходном файле многогранника и выполнения анализа видимости для этого многогранника с помощью алгоритма Робертса.

2.2. Проектирование программы

2.2.1. Формат файла сцены

В первой работе мы выбрали способ представления сцены, составленной из отрезков. Сейчас нам нужно расширить его для представления многогранников. Наиболее простой способ задания многогранника – задать его как набор граней (многоугольников). Продолжая использовать идею о разделении геометрической и топологической информации, логично сохранить способ представления набора точек, а многоугольник задавать как последовательность целых чисел, размещенных в одной строке файла. Каждое из этих чисел соответствует одной вершине многоугольника и представляет собой номер точки в списке точек. Единичный квадрат в плоскости XZ тогда может быть представлен как:

```
4
0.0 0.0 0.0
1.0 0.0 0.0
0.0 0.0 1.0
1.0 0.0 1.0
1
0 1 3 2
```

или

```
4
0.0 0.0 0.0
1.0 0.0 0.0
0.0 0.0 1.0
1.0 0.0 1.0
2
0 1 3
0 3 2
```

В последнем случае мы разбили квадрат на 2 треугольника. При таком подходе все преобразования, реализованные в задаче № 1, продолжают работать без каких-либо изменений. Для отображения многоугольников нужно внести очевидные изменения, т.е. добавить возможность рисования набора отрезков, заданного последовательностью номеров вершин.

2.2.2. Реализация алгоритма Робертса

Для корректной работы алгоритма Робертса необходимо вычислить внешнюю нормаль для каждой грани заданного пользователем многогранника. Имеет смысл разделить эту задачу на две части. Сначала вычислить какую-нибудь нормаль, а затем определить, является ли она внешней. Если да, то задача решена, в противном случае нужно изменить направление найденного вектора нормали на противоположное.

Найти какую-нибудь нормаль проще всего векторным произведением двух векторов, соответствующих двум последовательным ребрам исходного многоугольника. Если мы можем быть уверены, что два последовательных ребра неколлинеарные, то больше ничего не нужно. Если такой уверенности нет, то есть смысл проделать эту процедуру для всех пар последовательных векторов и выбрать векторное произведение с максимальным модулем. Такой подход все еще не приведет к результату, если многоугольник вырожден в отрезок, но в этом случае задача неразрешима в принципе, т.к. невозможно разумным образом определить понятие нормали для такого многоугольника.

Проверка является ли найденная нормаль внешней не может, очевидно, быть выполнена локально. Это значит, что невозможно сделать эту проверку, рассматривая только одну грань. Необходимо будет использовать другие вершины многогранника, т.е. вершины, принадлежащие другим граням. Если многогранник выпуклый, а алгоритм Робертса способен работать только с выпуклыми, то достаточно одной такой точки. Более точно, нам нужно найти любую вершину многогранника, не лежащую в плоскости рассматриваемой грани. Если считать, что никакие две точки в списке точек не совпадают, то нам нужно найти вершину, номер которой не содержится в списке номеров вершин рассматриваемой грани. Для этого просто будем перебирать номера всех вершин многогранника, для каждой из них перебирать все вершины текущей грани и сравнивать. Поиск закончен, когда очередная вершина многогранника не совпала ни с одной вершиной текущей грани. Эта процедура, конечно, может быть выполнена более эффективно, но это не является принципиальным в данном случае.

Пусть теперь $\vec{n}$ – найденный нами вектор нормали, v – одна из вершин текущей грани, p – найденная нами вершина многогранника, не принадлежащая текущей грани. Тогда очевидно, что нормаль является внешней если и только если $(p - v) \cdot \vec{n} < 0$.

После того, как внешние нормали найдены реализация алгоритма Робертса сводится к проверке знака скалярного произведения $\vec{n} \cdot \vec{e}$ для каждой грани. Здесь $\vec{n}$ – внешняя нормаль, а $\vec{e}$ – направление взгляда. В нашем случае вектор $\vec{e}$ имеет координаты $\begin{pmatrix} 0 & 0 & -1 \end{pmatrix}$, т.к. ось Z направлена на наблюдателя. Если это скалярное произведение отрицательно, то грань видима. Если положительно, то грань невидима.

2.3. Возможные вопросы

1. Если задать исходные точки так, чтобы многогранник оказался невыпуклым, то результат анализа видимости при каком-то ракурсе окажется неправильным. Почему это происходит? Какие именно вычисления в вашей программе приводят к такому результату?
2. Как ваша программа отличает внешнюю нормаль от внутренней?

ГЛАВА 3. ЛАБОРАТОРНАЯ РАБОТА № 3

3.1. Постановка задачи

Расширить возможности программы второй лабораторной работы возможностью выполнять анализ видимости с помощью алгоритма Z-буфера.

3.2. Проектирование программы

3.2.1. Формат файла сцены

Алгоритм Z-буфера, в отличие от алгоритма Робертса, способен обрабатывать сцену, составленную из произвольного набора многоугольников, а не только один многогранник. К счастью, способ представления многогранника, который мы использовали для решения задачи № 2, годится и для этого случая без каких-либо изменений. Однако, есть, все-таки, один нюанс. Алгоритм Z-буфера является растровым. Результатом его работы будет изображение закрашенных многоугольников, заданных в исходном файле. Если все эти многоугольники будут закрашены одним цветом, то невозможно будет отличить один от другого, а, следовательно, невозможно будет представить себе сцену и оценить правильность работы алгоритма анализа видимости. Более того, изображение будет одинаковым как с анализом видимости, так и без него. В рамках следующей, четвертой лабораторной работы мы научимся вычислять цвет для каждой грани в соответствии с заданным освещением. Тогда проблема отпадет сама собой. Чтобы избежать ее уже сейчас, можно либо выполнять эти две работы одновременно, либо задать в исходном файле свой цвет для каждой грани, либо присваивать эти цвета внутри программы по какому-нибудь правилу. Во втором случае придется внести небольшие изменения в структуру исходного файла, а именно, добавить возможность задать цвет для каждой грани.

3.2.2. Реализация алгоритма Z-буфера

Есть 2 пути, по которым можно пойти при выполнении этой работы. Первый – использовать готовый продукт, в котором алгоритм Z-буфера уже реализован (например, OpenGL). На этом пути объем работы по программированию будет существенно меньше, но придется самостоятельно освоить все, что касается основ работы с OpenGL.

Второй путь – самостоятельная реализация алгоритма. Обсудим его подробнее.

```
Алгоритм Z-буфера можно записать так:
A. Инициализировать Z-буфер и буфер кадра.
B. Для каждого многоугольника в сцене:
    1. Для каждого пикселя, принадлежащего проекции
                многоугольника на картинную плоскость
                (X, Y):
        A. Вычислить значение Z(X, Y).
        B. Если Z(X, Y) > Zb(X, Y):
            1. Заменить значение Zb(X, Y) на Z(X, Y).
            2. Заменить цвет элемента (X, Y) буфера кадра
                на цвет текущего многоугольника.
```

Здесь Zb(X, Y) – значение элемента (X, Y) из Z-буфера.

Обсудить реализацию имеет смысл для пунктов B.1 и B.1.A. Начнем со второго. Вычислить значение Z для плоскости при заданных X и Y проще всего, построив уравнение этой плоскости в виде $Ax + By + Cz + D = 0$ и подставив в него X и Y. Коэффициенты A, B и C в этом уравнении есть компоненты вектора нормали к плоскости. Вычисление этого вектора обсуждалось в задаче № 2. Для вычисления D можно подставить в качестве x, y, z координаты любой точки, лежащей в плоскости, например, одной из вершин. Рассмотрим еще вырожденные случаи. Формула для вычисления Z будет иметь вид:

$$Z = -\frac{A \cdot X + B \cdot Y}{D}$$

Особенность здесь возникает только при $D = 0$. Такая плоскость оказывается перпендикулярной к картинной плоскости и, следовательно, ее можно игнорировать, т.к. она на вносит никакого вклада в результирующее изображение. Рассмотрим теперь подробнее пункт B.1. Здесь должен быть использован какой-нибудь алгоритм растровой развертки (закраски) многоугольника. Например, алгоритм со сканирующей строкой:

```
A. Для каждой строки растра (Y):
    1. Для каждого ребра многоугольника:
        A. Найти точку пересечения ребра с горизонтальной
                прямой y = Y.
B. Упорядочить все найденные точки пересечения по
                возрастанию координаты X. Нумеровать,
                начиная с индекса 0.
C. Для каждой точки с четным индексом:
    1. Для каждого пикселя в текущей строке, лежащего
                между этой точкой и следующей:
        A. Передать пиксель для дальнейшей обработки в
                алгоритме Z буфера.
```

Особенностью работы этого алгоритма является его непредсказуемое поведение в случае, когда сканирующая прямая точно попадает в вершину многоугольника, т.е. координата Y одной из вершин многоугольника совпадает со значением Y для сканирующей прямой. Есть различные способы устранения этой непредсказуемости. В нашем случае проще всего пойти по пути небольших изменений значения Y сканирующей прямой, до тех пор пока оно не перестанет совпадать с координатой Y какой-либо вершины. При этом мы обязаны обеспечить, чтобы сканирующая прямая не вышла за пределы строки растра,

для которой она построена. Если сканирующая прямая изначально проводится в середине строки пикселей, то это значит, что суммарное изменение Y не может превышать половину высоты пикселя. Для того чтобы обеспечить выполнение этого условия, достаточно задать значение «небольшого изменения» меньшим, чем $\frac{H}{2(N+1)}$, где H – высота пикселя, а N – количество вершин многоугольника.

Реализация пункта А.1.А – это, конечно, просто нахождение точки пересечения прямой и отрезка. В данном случае его можно выполнить так. Сначала проверить лежит ли значение Y сканирующей прямой между значениями Y начальной и конечной точки отрезка. Если да, то вычислить значение X точки пересечения по формуле:

$$X = X_0 \cdot (1 - d) + X_1 \cdot d$$

$$d = \frac{|Y_0 - Y|}{|Y_0 - Y_1|}$$

где X_0, Y_0, X_1, Y_1 – координаты X и Y начальной и конечной точек отрезка. Единственная особенность в этой формуле возникает при $Y_0 = Y_1$, т.е. когда отрезок горизонтальный. В нашем случае неприятности, связанные с этой особенностью, возникнуть не могут, т.к. либо этот отрезок не пройдет входной тест (Y лежит между ...), либо будет распознана ситуация попадания сканирующей прямой в вершину и значение Y этой прямой будет изменено, после чего отрезок уже точно не пройдет тест.

3.3. Возможные вопросы

1. Что произойдет, если вершины одного многоугольника не лежат в одной плоскости?
2. Как в программе реализовано эффективное использование всего диапазона изменения Z?

ГЛАВА 4. ЛАБОРАТОРНАЯ РАБОТА № 4

4.1. Постановка задачи

Дополнить возможности программы автоматическим определением цвета для каждой грани в соответствии с простейшей моделью освещения и обеспечить пользователю возможность изменять коэффициенты зеркального и диффузного отражения и степень зеркальности.

4.2. Проектирование программы

Эта задача, наверное, самая простая из четырех. Необходимо обеспечить вычисления по заданным формулам модели освещения. Исходными данными для этих расчетов будут вектор нормали к плоскости многоугольника, направление взгляда и направление света. Задание не требует возможности управлять двумя последними параметрами, поэтому их можно сделать одинаковыми и равными $\begin{pmatrix} 0 & 0 & -1 \end{pmatrix}$. Это будет означать что пользователь смотрит вдоль оси Z и свет падает в таком же направлении.

Рассеянное отражение Интенсивность рассеянного отражения равна:

$$I_d = I \cdot k_d \cdot \cos\theta; 0 \leq \theta \leq \frac{\pi}{2}$$

Из формулы видно, что значение угла θ нам не нужно, достаточно получить значение $\cos\theta$. Скалярное произведение векторов выглядит следующим образом:

$$\vec{a} \cdot \vec{b} = |\vec{a}| \cdot \left|\vec{b}\right| \cdot \cos\theta \Rightarrow \cos\theta = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}| \cdot \left|\vec{b}\right|}$$

Пусть $\vec{a}$ – это вектор падающего на поверхность луча, $\vec{b} = -\vec{n}$ – это вектор, обратный вектору нормали, тогда $\cos\theta$ – это косинус угла между падающим лучом и нормалью к поверхности. Получается, что вычисление значения угла не является необходимым.

Зеркальное отражение Интенсивность зеркального отражения равна:

$$I_m = I \cdot k_m \cdot \cos^n \beta$$

где β – это угол между лучом зрения и отражённым лучом.

Получение вектора отражённого луча Зная вектор нормали $\vec{n}$ и вектор падающего луча $\vec{L}$ можно получить вектор отражённого луча по следующей формуле:

$$\vec{S} = \vec{L} - 2 \cdot \frac{\vec{n} \cdot \left(\vec{n} \cdot \vec{L}\right)}{|\vec{n}|^2}$$

Особенностей в этих формулах быть не может, т.к. единственный знаменатель – квадрат модуля вектора нормали. Рассматривая алгоритм Робертса, мы уже обсуждали, как добиться того, чтобы этот вектор не был нулевым. Подумать нужно только о знаках $\cos\alpha$ и $\cos\beta$ в этих формулах.

Ясно, что знак $\cos\alpha$ зависит от выбора направления вектора нормали к поверхности. Если мы хотим, чтобы $\cos\alpha$ всегда был неотрицательным, то вектор нормали должен быть направлен так, чтобы скалярное произведение вектора взгляда на вектор нормали было неотрицательным. Точно такой же эффект мы получим если просто подставим в формулу модуль $\cos\alpha$. Такой подход часто называют «односторонним» представлением. Здесь имеется в виду, что обе стороны грани являются абсолютно равноправными, т.е. вычисленная интенсивность не зависит от того, с какой стороны наблюдатель смотрит на грань. Двустороннее представление позволяет различать разные стороны одной и той же грани и, соответственно, вычислять для них различные атрибуты. В этом случае для положительных значений $\cos\alpha$ используются одни коэффициенты, а для отрицательных – другие.

Со знаком $\cos\beta$ ситуация другая. Направление вектора отраженного луча $\vec{S}$, вычисленного по приведенной выше формуле, не будет зависеть от направления нормали. Точнее, не будет зависеть от того, какую из двух возможных

нормалей мы выберем. Знак будет зависеть от того, по одну сторону от плоскости грани находятся наблюдатель и источник света или по разные стороны. Здесь тоже можно использовать различные стратегии по определению атрибутов для различных случаев взаимного расположения наблюдателя и источника света. Например, можно считать, что если наблюдатель и источник находятся по разные стороны, т.е. знаки скалярных произведений вектора направления взгляда на вектор нормали и вектора направления света на вектор нормали разные, то этот источник света не влияет на наблюдателя и соответствующее значение интенсивности равно нулю.

Учебное электронное текстовое издание

Кац Евгений Исакович
Китаев Андрей Михайлович

КОМПЬЮТЕРНАЯ ГРАФИКА

Методические указания
к лабораторным работам

Редактор *А.В. Ерофеева*
Компьютерная вёрстка *А.М. Китаев*

Рекомендовано Методическим советом
Разрешен к публикации 14.01.2016
Электронный формат – pdf
Объём 0,9 уч.-изд.л.

Уральский федеральный университет имени первого Президента России Б.Н.Ельцина

620002, Екатеринбург, ул. Мира, 16

Информационный портал УрФУ
http://study.urfu.ru